

DSA Coach AI

Code Execution Agent — Project Description & Scope

Project focus: Runtime validation of candidate-submitted DSA programs using a dedicated Code Execution Agent.

1. Project Overview

DSA Coach AI is an AI-powered DSA coding platform designed to help candidates prepare for coding interviews. This document focuses only on the Code Execution Agent. Unlike an LLM that only analyzes source code, the agent provides actual runtime evidence by executing submitted programs, capturing output, detecting runtime errors, and handling execution timeouts.

2. Problem Statement

Candidates need to know not only whether their code appears logically correct, but whether it actually runs. Static or LLM-based analysis can miss runtime problems such as undefined variables, exceptions, invalid imports, and infinite loops. The Code Execution Agent addresses this by executing submitted programs and returning structured runtime feedback.

3. Main Objective

The primary objective is to execute candidate-submitted code and provide structured runtime feedback, including execution status, program output, runtime errors, and timeout information.

4. Project Scope

The system accepts a problem statement, programming language, candidate approach, and source code. The Code Execution Agent validates the language, creates a temporary source file, executes the program, captures standard output and standard error, detects failures and timeouts, cleans up the temporary file, and stores the result in shared state.

5. Code Execution Agent

The Code Execution Agent is responsible only for runtime execution. It does not evaluate algorithmic optimality, communication, or learning requirements. Its job is to run the candidate's code and report what happened.

6. Execution Workflow

Candidate Code → Language Check → Temporary File → Write Code → Execute Program → Capture Output/Error → Timeout Check → Execution Result → Shared State.

7. Temporary File and Execution

The candidate source is written to a temporary Python file using `tempfile` and executed through Python's subprocess mechanism. Standard output, standard error, and the process return code are captured.

8. Runtime Error Detection

If the submitted program raises an exception, the agent captures the runtime error and returns it as structured feedback rather than allowing the backend workflow to fail.

9. Timeout Detection

A timeout prevents an infinite or excessively long-running program from blocking the backend. The current prototype uses a five-second execution limit.

10. Cleanup

The temporary source file is removed after execution, including when an exception or timeout occurs.

11. Execution Result

The agent returns a structured dictionary containing status, output, and error. Typical statuses are passed, failed, timeout, and unsupported.

12. Shared State

The result is stored in state["execution_result"]. This field is represented as a Dict in DSAAgentState so the result can be consumed by other backend components.

13. FastAPI Integration

React communicates with FastAPI. The frontend submits candidate data to the backend; FastAPI initializes the workflow and the Code Execution Agent produces the runtime result, which is returned through the API.

14. Frontend Integration

The React frontend provides fields for the problem statement, programming language, approach, and source code. It displays the returned execution status, output, or error.

15. LangGraph Integration

LangGraph provides agent orchestration and shared state. For the isolated Code Execution Agent scope, the simplified workflow is START → Code Execution Agent → END.

16. Technology Stack

Frontend: React, JavaScript, Vite. Backend: Python, FastAPI, Uvicorn. Agent orchestration: LangGraph. Execution: Python subprocess and tempfile. Configuration: python-dotenv. Future containerization: Docker.

17. Why a Code Execution Agent?

An LLM can reason about whether code appears correct, but runtime execution provides direct evidence about what actually happened. Runtime validation can distinguish a logically plausible solution from a program that genuinely executes successfully.

18. Security Considerations

The current implementation is a local/prototype execution component. Arbitrary user code should not be directly exposed to an untrusted public environment. Production deployment should isolate execution using a secure sandbox or container with CPU, memory, timeout, filesystem, network, and process restrictions.

19. Future Scope

Future enhancements include multi-language execution such as Java, C++, JavaScript, and Go; automated test-case execution; expected-versus-actual output comparison; execution-time and memory metrics; secure sandboxing; and a dedicated cloud code-execution service.

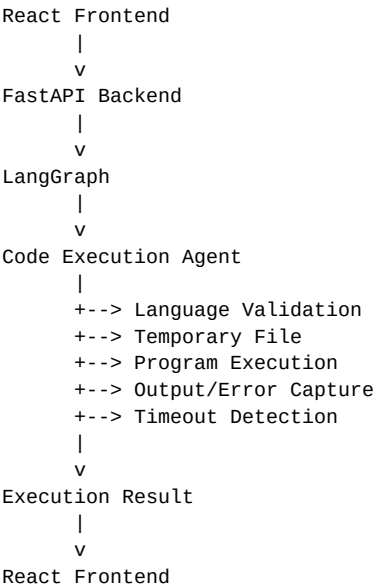
20. Key Advantages

The architecture provides runtime validation, structured error reporting, timeout protection, modular agent design, FastAPI integration, and a path toward multi-language and secure cloud-based execution.

21. Interview Explanation

The Code Execution Agent provides runtime validation for candidate-submitted DSA code. It receives source code and programming language through the LangGraph state, creates a temporary source file, executes the program using a subprocess, captures standard output and runtime errors, and applies a timeout to prevent infinite execution. It stores a structured execution result containing status, output, and error in shared state. For production, the execution should be isolated in a secure sandbox with CPU, memory, filesystem, network, and timeout restrictions.

22. Architecture



23. Example Execution Results

Situation	Status	Output	Error
Successful	passed	[0, 1]	
Runtime failure	failed		Runtime exception
Long-running code	timeout		Exceeded 5 seconds
Unsupported language	unsupported		Language not implemented

24. Example Result

```
{
  "status": "passed",
  "output": "[0, 1]",
  "error": ""
}
```

25. Project Status

Component	Status
React Frontend	Implemented
FastAPI Backend	Implemented
LangGraph Integration	Implemented
Code Execution Agent	Implemented
Python Execution	Implemented
Runtime Error Handling	Implemented
Timeout Handling	Implemented
Temporary File Cleanup	Implemented
Multi-language Execution	Future
Secure Sandbox	Future
Automated Test Cases	Future
Cloud Deployment	Future